

Bases de Datos 2

Resumen teorías

Clase 1 - Introducción	4
Bases de datos relacionales	4
Orientación a objetos	4
Posibles problemas	4
Memoria	4
Archivos	4
BBDD relacionales	4
Diferencia de impedancia	5
Principio de ortogonalidad	5
Principio de independencia	5
Persistencia por alcance	5
Clase 2 - Mapeo Objeto-Relacional	6
Transacciones	6
Persistencia por alcance	6
Mapeo	7
Ids	7
Atributos	7
Relaciones	7
Jerarquías	7
Clase 3 - Hibernate y JPA	9
Mapeos	9
Reglas a seguir para el mapeo de POJOs	9
Mapeos en JPA	9
Entidades	9
Jerarquías	10
SINGLE_TABLE	10
JOINED	10
TABLE_PER_CLASS	11
Relaciones	11
CascadeType	11
FetchType	11
ManyToMany	12
OneToMany	12
DTOs	12
Migrations	12
Concurrencia	12
Locking pesimista	13
Locking optimista	13
Clase 4 - Proxys y caching en Hibernate	14

Proxies	14
Caching	14
Nivel 1	14
Nivel 2	14
Clase 5 - Consultas y patrones de persistencia	15
Consultas	15
Patrones de persistencia	15
DAOs	16
ORMs	17
Repository	17
Clase 6 - NoSQL	19
Ventajas y desventajas de los RDBMS	19
NoSQL	20
Agregaciones	20
Transacciones	20
Familias	20
Relaciones	20
Grafos	21
Clase 7 - MongoDB	22
Características	22
Almacenamiento	22
Schemaless	22
Analogía con RDBMS	22
Relaciones	23
Índices	23
Aggregation Framework	23
Clase 8 - MongoDB distribuido	24
Objetivos	24
Ventajas	24
Réplica sets	24
Sharding	24
Consistencia	25
Inconsistencias de escritura	25
Inconsistencias de lectura	26
Teorema CAP	27
Quorum	27
Replicación	27
Clase 9 - Familia de columnas	29
Cassandra	30
Ejemplo de Blog	31
Keyspace	32
Particiones y distribución	33
Clase 10 - Data warehouse	35
Arquitectura	35

BBDD vs DW	37
Tablas de un DW	37
Hechos (medidas)	37
Dimensiones	37
Modelos	38
Estrella	38
Copo de nieve	40
Galaxia/Constelación	41
Data Mart	42
Data Lake	42

Clase 1 - Introducción

Bases de datos relacionales

SQL no sólo es para hacer consultas, sino que también describe la BD, sirve para hacer backups, maneja usuarios, entre muchos otros aspectos

SQL es muy similar entre los distintos RDBMS lo cual es poderoso

SQL es un estándar, lo cual permite migrar fácilmente

Orientación a objetos

Está muy presente en la industria actual

Así como SQL permite para almacenar y estructurar información, OO permite que dichos datos actúen

Posibles problemas

¿Dónde puedo guardar los objetos?

Memoria

La memoria, además de no permitir persistencia, no permite distribución por lo que no va a ser ideal

Archivos

Son lentos, no tienen índices ni transacciones...

BBDD relacionales

Tiene muchas herramientas para facilitar la persistencia (índices, transacciones, ...)

Pero existe lo que se llama diferencia de impedancia con los sistemas OO

Diferencia de impedancia

- En OO existen cosas que en las BBDD relacionales no, por ejemplo:
 - Jerarquías
 - Comportamiento
 - Permite modelar objetos sin límite aparente
- En las BBDD relacionales existen cosas que no existen en los sistemas OO:
 - Transacciones

Se podría pensar que se está volviendo al paradigma procedural, al separar el comportamiento de los datos, pero esto no es así:

- Seguimos programando en OO
- Y buscamos olvidarnos de la BD subyacente

Hay dos principios que refuerzan esto: el de ortogonalidad y el de independencia

Principio de ortogonalidad

Todo objeto debe poder ser persistido más allá de su tipo (más allá de los transient objects para los cuales es una elección no persistirlos)

Principio de independencia

El sistema OO debe saber lo menos posible sobre la tecnología de persistencia subyacente

Persistencia por alcance

Si guardo un objeto persistente que conoce a otro (que no es transient), al persistir el primero, se persiste el segundo también

Esto reduce el número de saves

Sirve para fomentar el principio de independencia

Pero..., como un sistema OO no tiene bordes aparentes, podrían suceder saves en cadena que traigan problemas (y algo similar sucede al recuperar objetos de la BD)

Clase 2 - Mapeo Objeto-Relacional

Por el principio de independencia, la BD tiene que adecuarse al mundo OO, pero al incorporar la persistencia, el mundo OO tendrá que adquirir aspectos de BBDD:

- Para cumplir las propiedades ACID necesitamos tx
- Necesitamos performance al costo de otras propiedades
- Problemas con el scope de las tx...

Transacciones

En OO se verán así:

- Se delimitan con begin/commit
- No se pueden anidar
- Problema de locking, no se puede dejar sin acceso a otros por mucho tiempo

Persistencia por alcance

Si tenemos un objeto persistente (no transient) que conoce a otro también persistente (no transient), al guardar el primero la acción se propagará al otro (dependiendo de la configuración), reduciendo la cantidad de saves a usar

Un problema que puede traer es que la cadena de propagación podría ser muy larga o generar un bucle

Una posible implementación es el uso del patrón Proxy, es decir, tener un objeto que pueda recibir los mensajes del objeto real, pero que hasta que no se necesita, no se obtenga de la BD

Con esta implementación, cada objeto pasa por distintos estados (transient, persistent, detached, deleted, ...), lo que puede agregar nuevos problemas; por ejemplo, si se intenta usar un objeto lazy fuera del contexto de persistencia

Mapeo

Ids

Se deben usar Ids como clave primaria en las BBDD, los cuales deben ser únicos y no deben estar relacionados con el dominio

Hay dos estrategias para manejarlos:

- Por la BD (auto incremental, long, ...)
- Por la aplicación (ventaja: no requiere esperar el commit)

Atributos

En OO todo es un objeto, mientras que en los RDBMS, existen muchos tipos de atributos

Al usar mapping, se pueden diferenciar los tipos atómicos de las relaciones con otros objetos

Relaciones

Mapeo de los diferentes tipos de relaciones:

- 1 a 1: simple
- Unidireccionales 1 a N, se da como se vio según la persistencia por alcance
- Bidireccionales 1 a N. Hay que tener en cuenta quién va a ser el owning side, es decir, quien tiene la FK
 - Por ejemplo, Cliente > Reservas. Reserva sería owner porque guarda FK. En el modelo esto hace que cliente.agregarReserva(cena) NO guarde la relación

Jerarquías

- Una tabla por cada clase concreta:
 - No tener la clase padre
 - Se hace una unión de los atributos de la clase padre y cada uno de los hijos
- Una tabla por cada clase del modelo:
 - Se mantienen todas las tablas

- Se deben usar JOINS para obtener todos los datos de una de las entidades concretas
- Una tabla para toda la jerarquía:
 - Implica tener campos en null (los que son de un tipo diferente al de la relación actual)
 - Implica un discriminante que indique el tipo de entidad

Clase 3 - Hibernate y JPA

Es el ORM por referencia de JPA

Soporta jerarquías, polimorfismo, relaciones, transacciones, laziness, HQL, etc.

Mapeos

Se pueden definir mediante XML, lo cual es más tedioso que usar annotations, pero mejor para la independencia, ya que mantiene el código de las clases del modelo más limpio (menos “contaminado” por el código de persistencia)

Por eso decimos que las annotations permiten un mapeo más simple pero más intrusivo, al romper el principio de independencia

Reglas a seguir para el mapeo de POJOs

- Implementar un constructor sin argumentos
- Proveer un identificador
- Clases no-finales (no pueden ser heredadas). Esto es así ya que Hibernate usa el patrón proxy para representar objetos de la BD, y para eso el proxy debe heredar la clase de mi modelo
- Setters y getters para los campos persistentes

Mapeos en JPA

Entidades

Las entidades en JPA son POJOs persistentes

Por default en JPA se persisten todos los atributos, a menos que se marquen como `@Transient`. Un ejemplo de Convention over Configuration (al contrario que Hibernate con XML)

Las entidades tienen opciones para configurar, como el nombre de la tabla (útil para evitar palabras reservadas del DBMS)

Los atributos también, pueden especificarse propiedades de la columna, restricciones, etc.

También se puede ligar @Column a un getter/método

Jerarquías

SINGLE_TABLE

```
@Entity(name = "clientes") 7 usages
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo")
public abstract class Cliente {
    // ...
}
```

Se usa una sola tabla para toda la jerarquía

Cada tupla de la tabla será una entidad completa

Necesita un discriminante para diferenciar los distintos tipos de entidades (podría ser un entero para ser más performante)

Tiende a ser más útil cuando la clase padre es abstracta

Tiende a ser más performante al no requerir joins

Desde el punto de vista de la normalización no es tan óptima

JOINED

```
@Entity(name = "clientes") 7 usages
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Cliente {
    // ...
}
```

Hay una tabla con los atributos en común y luego una tabla más por cada subclase donde están los atributos que no se comparten y un id a la tupla en tabla padre

Tiende a ser más útil cuando la clase padre no es abstracta

No es tan performante como SINGLE_TABLE al requerir joins

Es el tipo de jerarquía más óptimo desde el punto de vista de la normalización

TABLE_PER_CLASS

```
@Entity 8 usages
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Cliente {
```

Hay una tabla por cada clase concreta

No se puede usar si la superclase no es abstracta

La superclase se marca como Entity pero no se indica nada relacionado a la tabla, dado que no se generará

Relaciones

CascadeType

Establece la persistencia por alcance

Por defecto nada se propaga en JPA

Puede ser ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH

El valor a elegir dependerá del uso

FetchType

El tipo de Fetch puede ser EAGER o LAZY, cada uno puede ser más o menos performante según el caso

El valor a elegir dependerá del uso

ManyTo[Many|One]

No requiere especificación de tipo (como Hibernate)

Por default se crea una tabla intermedia

OneToMany

Mapping por default con tabla intermedia

Se puede evitar la tabla intermedia con el uso de JoinColumn, que indica la columna de la otra tabla que referencia a la actual:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private List<PostComment> comments = new ArrayList<>();
```

DTOs

No puedo leer objetos fuera de una tx, qué hago?

DTOs. Según Fowler: "An object that carries data between processes in order to reduce the number of method calls."

Se puede aprovechar el hecho de crear otra clase y cambiar el formato, agregar atributos externos, etc.

Migrations

Al usar ORMs tenemos que considerar la evolución

Existe el concepto de migraciones - pensarlo como el historial "commits" en la estructura de la BD

Concurrencia

¿Cómo se manejan los problemas de concurrencia?

Se observa el @Version al escribir, - si el nro es el mismo de la base, ok - si no, tengo una versión vieja << hasta ahí llega JPA - solo eleva excepción

Locking pesimista

Se toma la BD, hace la transacción completa y cuando termina libera la BD

Locking optimista

Se hace la query y se ve si funcionó o no

De esta manera, otra sesión puede tomar el mismo objeto con el que se está trabajando y modificarlo mientras se trabaja

Clase 4 - Proxys y caching en Hibernate

Proxies

Qué pasa si...

1. Abro una sesión, una tx y guardo un objeto “reserva” relacionado a otro, “cliente” de forma LAZY
2. Abro otra tx y recupera “reserva”
3. Pido el “cliente” a la reserva recuperada
4. Cierro sesión

¿El cliente, debería ser un proxy?

¿O un objeto del modelo?

¿Y si hago los pasos 2-3 en una nueva sesión?

Caching

Nivel 1

Activado por default

Depende de cada sesión

Cuando una entidad se carga o actualiza por primera vez en una sesión, se almacena en la caché a nivel de sesión

Solicitudes posteriores de la misma entidad en la misma sesión se sirven desde la caché

Se minimizan así accesos a la BD

Nivel 2

A nivel de SessionFactory

Si no se halla un objeto en la Caché L1, se busca en Caché L2

Es un hotspot del framework Hibernate; no viene implementada, pero hay productos que lo hacen

Se pueden cachear resultados de consultas HQL

Hay implementaciones para descargar

Clase 5 - Consultas y patrones de persistencia

Consultas

Hibernate ofrece:

- SQL:
 - Es rápido, pero es un string (debugging complejo) y está atado a la BD 100% (tablas en vez de clases mapeadas)
 - Puede formar objetos con la respuesta
- HQL:
 - Clases en el FROM
 - No puedo usar mensajes, solo atributos
 - Sigue siendo un string
- Criterios:
 - Se construye la query en objetos
 - Es más debugger-friendly
 - Más lento

A mayor abstracción, menor performance y mayores limitaciones

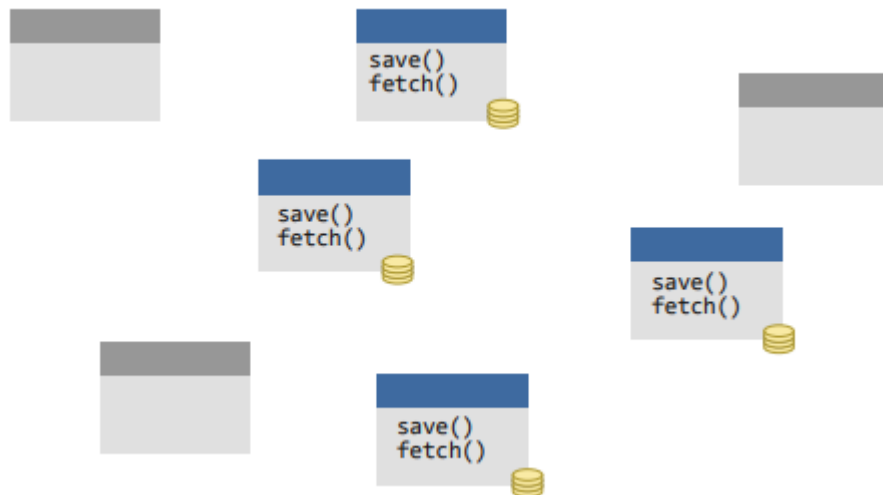
Patrones de persistencia

¿Cómo persistir un modelo de OO si no conociéramos los ORM?

Caso naïve - cada clase se ocupa de su “guardar”, “obtener”, etc.

Problemas:

1. Necesito una instancia de cada una para que funcione
2. Acoplamiento con la tecnología de persistencia

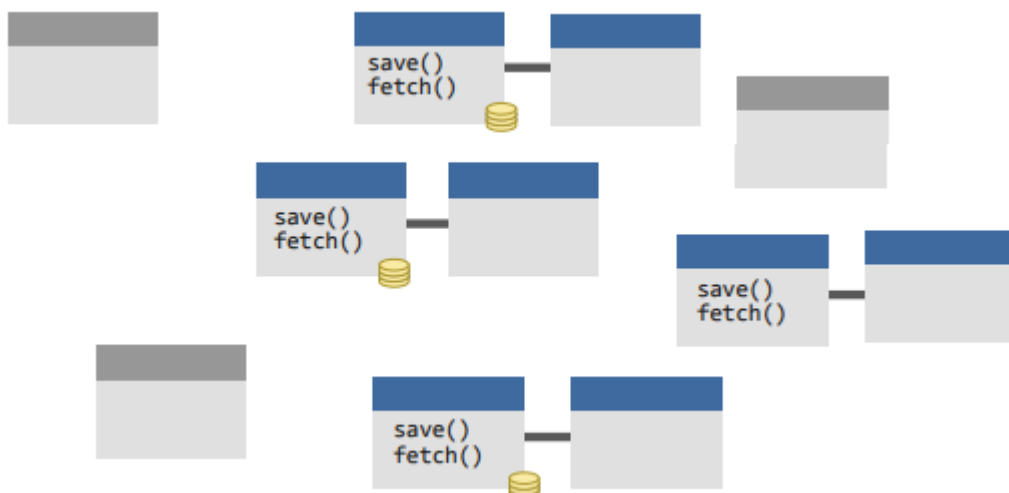


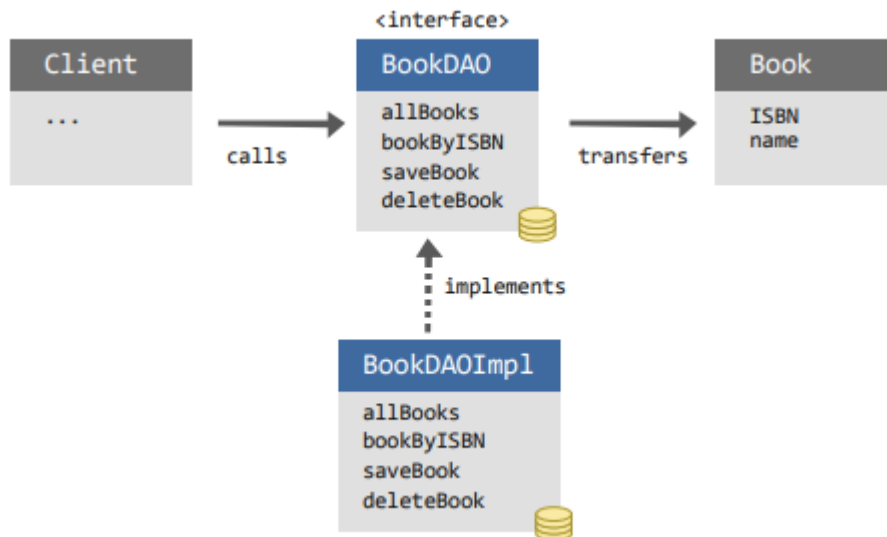
DAOs

Establece que por cada objeto de dominio persistente debe existir una clase que persiste y recupera sus instancias

Problemas:

1. Granularidad/relaciones entre DAOs de clases relacionadas
2. “Mudanza” del comportamiento de negocio a la capa de persistencia/el modelo queda anémico
3. Acoplamiento





ORMs

Aparecen los ORMs

¿Dónde iría el ORM en este esquema?

Se solía poner en el DAO, cosa que desaprovecha las capacidades de todo ORM

Repository

El patrón Repository ya no representa la capa de persistencia, sino la colección de objetos persistentes

Se enfoca en el "fetch"

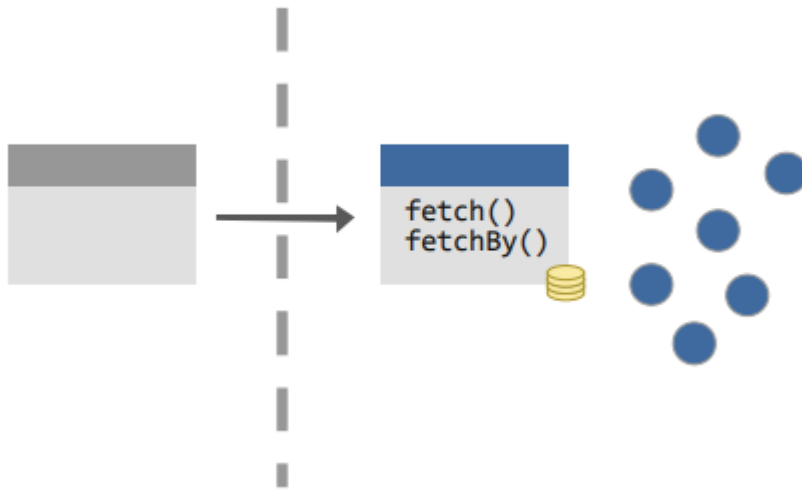
Aprovecha los updates de ORM

De esta manera, los save (guardar y actualizar) y deletes son genéricos, ya que el ORM conoce las propiedades de cada objeto, por lo que solo es necesaria la lógica para hacer los distintos tipos de fetch que sean necesarios

Puede emular las colecciones de Smalltalk y sus filtros y usos de bloques (ej. `findByEmail`, `getUsersCount`)

Puede complicarse con el tipado estático

La implementación de puede volver repetitiva



Clase 6 - NoSQL

Ventajas y desventajas de los RDBMS

Los RDBMS tienen muchas ventajas:

- Estandarización
- Buen manejo de concurrencia
- Simplicidad del modelo relacional
- SQL como unificador
- Las BBDD Relacionales juntan y resignan todo al costo de la consistencia
- Al tener muchos años de uso están optimizadas y con muy pocos errores

Pero también tienen limitaciones:

- Diferencia de impedancia: el modelo suele ser diferente de las estructuras usadas en memoria (OO). Lo que hace que tomemos decisiones de modelaje en base a futuro rendimiento, deformando el modelo que habíamos imaginado
- Integración:
 - Problema: muchas aplicaciones trabajan con un solo conjunto de datos
 - Se puede trabajar con una sola BD, pero hay que coordinar las aplicaciones
 - Esto puede ser complicado, pero:
 - Se pueden separar las aplicaciones y comunicarlas con Web Services o APIs REST
 - Cada aplicación puede elegir su propia BD
- Escalabilidad:
 - En el mundo actual, la escala de los datos generados es mucho mayor que anteriormente. Sin embargo, no se pueden crear servers monolíticos tan grandes, por lo que se deben usar clusters para escalar horizontalmente. El problema es que las RDBMS no está pensadas para eso

Resumen: por qué mirar más allá de las BBDD relacionales. Aunque las RDBMS aportan estandarización, buen manejo de concurrencia y un modelo simple, sufren por la “diferencia de impedancia” con los modelos en memoria, además de retos de integración y,

sobre todo, de escalabilidad: en el mundo actual de grandes volúmenes, escalar horizontalmente con clusters no es su punto fuerte

NoSQL

Las NoSQL proponen BBDD sin (o con menos) SQL, sin esquema rígido, pensadas para altos volúmenes y clustering, muy alineadas con la Web y el open source; fomentan además la persistencia políglota (elegir la BD según la aplicación)

Agregaciones

Agregaciones (Aggregates) como unidad de diseño. El modelado se centra en cómo se accede a los datos y en agrupar lo que suele consultarse junto dentro de la misma agregación, aceptando cierta duplicación (p.ej., direcciones repetidas entre “customer” y “order”) si mejora las lecturas. El límite del agregado se diseña pensando en las consultas reales y en despliegues en clusters

Por ejemplo, si necesitamos frecuentemente consultar reservas hechas, puede no ser una buena idea embeberlas en el cliente

Transacciones

Transacciones en NoSQL. No es que “no haya transacciones”: suelen existir, pero acotadas al agregado (granularidad distinta a las relacionales)

Familias

Hay 3 principales familias de NoSQL (todas son escalables):

1. Orientadas a documentos (MongoDB)
2. Clave-valor (Redis, DynamoDB):
3. Columnas (Cassandra)

Relaciones

Se pueden realizar relaciones entre agregaciones, pero:

- Las BD NoSQL no sabe de esto

- Las transacciones tienen alcance dentro de la agregación
- No son tan buenas las relaciones complejas
- Rápidamente se pueden complicar al sumar JOINS

Grafos

Pensadas para consultas con muchos saltos (redes sociales, conocimiento, web semántica)

Suelen requerir arquitectura más monolítica, tienen inserts más caros y queries baratas

Clase 7 - MongoDB

Características

- Base de datos NOSQL orientada a documentos
- No maneja la noción de esquemas
- Carece de un lenguaje DDL
- El formato de almacenamiento es BSON (Binary JSON)
- BSON guarda tamaños de campo para mayor velocidad de lectura
- Schemaless

Almacenamiento

Cuando se crea un documento, MongoDB asigna espacio extra (padding) para futuros cambios sin moverlo (sacrificio de espacio por performance)

Si crece demasiado, se mueve a otro lugar dejando una tombstone → posible fragmentación

El motor WiredTiger comprime (zlib o snappy) y compacta en background

Schemaless

No tiene una estructura definida

Esto da flexibilidad al desarrollar

Pero también trae riesgo de inconsistencias y documentos muy grandes

Analogía con RDBMS

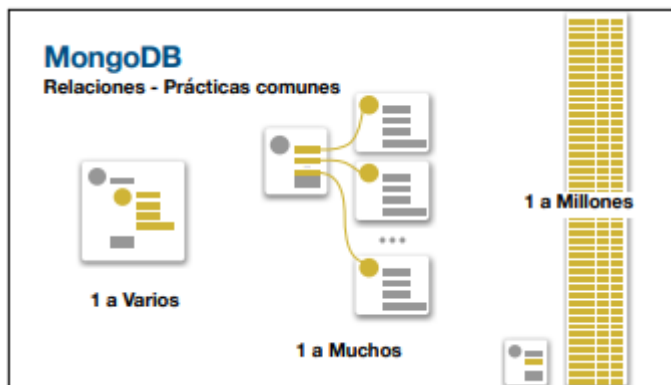
- BD → Base de datos
- Tabla → Colección
- Fila → Documento
- Columna → Campo

Relaciones

3 tipos:

- 1 a 1/pocos: documentos embebidos
- 1 a muchos: lista de referencias al otro documento
- 1 a millones: cada documento referenciado tiene una referencia al padre

Las restricciones las da Mongo ya que un documento no puede exceder los 16 MB de tamaño



Índices

Son a nivel de colecciones

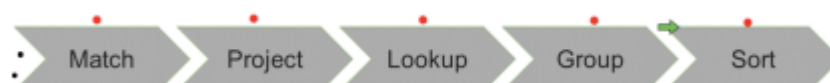
Se pueden hacer en el background sin bloquear la BD

Sobre `_id` es automático (único e ineliminable)

Tipos: simple, compuesto, geoespacial (2dsphere)

Aggregation Framework

Procesamiento de consultas complejas por etapas (pipeline), similar a operaciones en cadena



Clase 8 - MongoDB distribuido

Objetivos

Escalar horizontalmente, aumentar capacidad y disponibilidad

Ventajas

- La carga se distribuye horizontalmente en todo el cluster
- Capacidad de almacenamiento
- Si el espacio requerido aumenta, simplemente se agregan nuevos nodos al cluster
- Alta disponibilidad
- Aún cuando algún nodo no funcione, se puede seguir resolviendo parcialmente el pedido

Réplica sets

Grupo de nodos con la misma información

Mantienen la consistencia interna

Tienen failover automático

Configuración: se pone uno como primario preferentemente (después se administra solo)

Cuando el nodo primario se cae, los nodos de recovery negocian cuál será el nuevo primario. Cuando vuelve, se pone él mismo como recovery. Por esto, todos los nodos deben tener la capacidad de ser nodo primario

Sharding

Mongo particiona colecciones por shard key (que es uno o más campos inmutables)

Solamente puede existir una shard key por colección

Divide el espacio de valores en chunks

Los chunks se mueven dinámicamente para balancear carga

Chunks y Shards NO es 1 a 1. Pueden haber varios Chunks en un solo Shard

Se intenta distribuir los Chunks homogéneamente. Es más, los Chunks se mueven de Shard a Shard

Elección de shard key:

- Alta cardinalidad (muchos valores distintos).
- Distribución pareja.
- No crecer de forma monotonica (evitar concentrar inserciones en un shard)

Cada "shard" contiene un subconjunto de la información partida

A cada parte se la distribuye como "replica set"

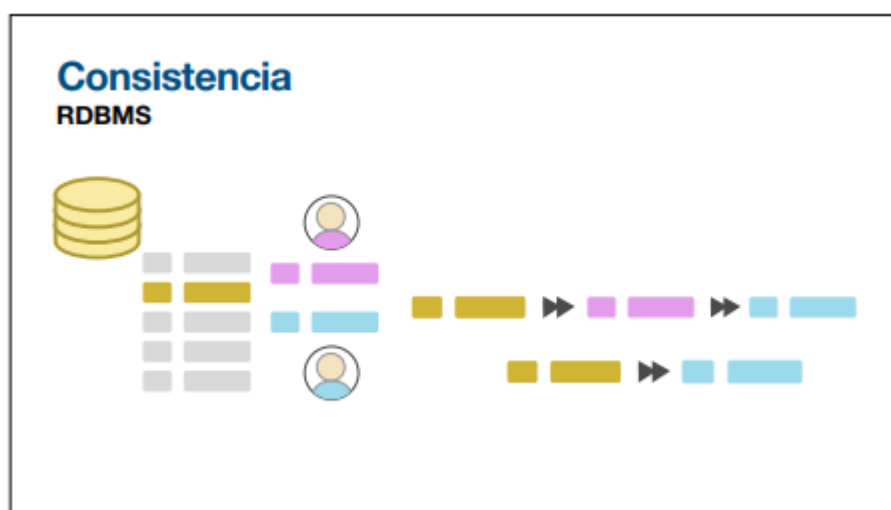
Mongos: Router para consultas hacia el shard correspondiente

Config servers: Guardan metadatos y configuración del cluster

Consistencia

Limitaciones de los RDBMS: buscan consistencia estricta a todo costo → problemas de latencia y disponibilidad en clusters

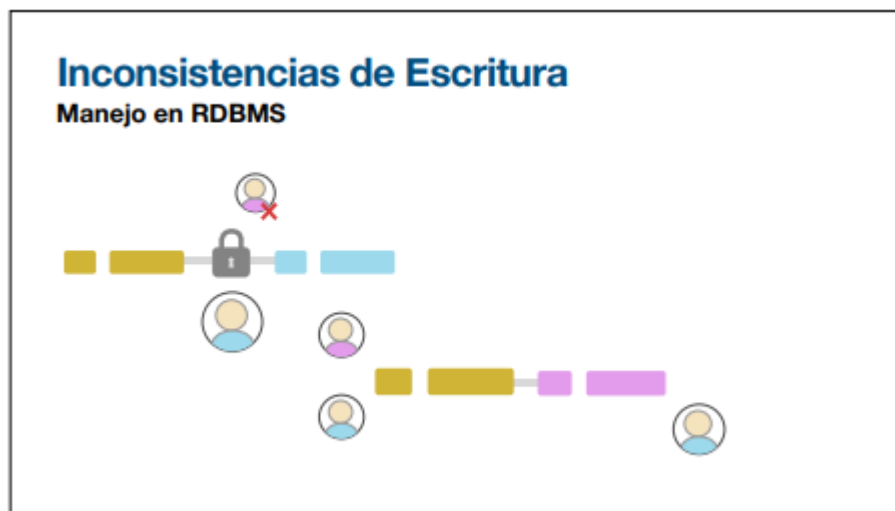
Inconsistencias de escritura



Hay una edición que se pierde: la que se aplica finalmente debía sobrescribir el estado original (amarillo), sin embargo termina sobrescribiendo el intermedio (rosa)

Para evitar esto, hay dos soluciones clásicas:

- Locking pesimista:
 - Previene el conflicto
 - En el ejemplo, se serializan las ediciones pero la primera obtiene el lock e impide la segunda
- Locking optimista:
 - Deja ocurrir el conflicto, pero actúa en consecuencia
 - Un ejemplo sería actualización condicional, donde se verifica que el valor no haya cambiado desde la última lectura antes de actualizar



Inconsistencias de lectura

Dos entidades deben mantener la consistencia

Actualización en etapas podría dejar una ventana inconsistente

Para evitar este escenario clásico se utilizan las tx

Las BBDD NoSQL (excepto grafo) tienen tx a nivel de agregación

Ejemplo, una habitación de hotel:

- Dos personas conversando por teléfono para tomarla, una en Londres y otra en Boston
- Un 3ro la toma desde Bombay
- Al mismo tiempo, la persona de Londres la ve libre y la de Boston tomada > las réplicas geográfica pueden tardar en actualizarse

Teorema CAP

“Given the three properties of Consistency, Availability, and Partition tolerance, you can only get two.”

Availability: if you can talk to a node in the cluster, it can read and write data

Partition tolerance: the cluster can survive communication breakages that separate the cluster into multiple partitions unable to communicate with each other

Un solo server puede ser CA, dado que no se va a partir. Los RDBMS son generalmente así

Se podría generar un cluster CA - si ocurre una partición, se apagan todos. De esta forma “Availability” según CAP se cumple igual

Todo radica en la P: cuando las particiones ocurren, hay que sopesar C y A

Quorum

¿Cuántos nodos necesito para obtener consistencia?

Si hay un master, uno

Si es P2P:

- Escritura:
 - No hace falta que todos admitan una escritura, sólo la mayoría
 - El quórum de escritura debe ser mayor a la mitad: $w > n/2$
- Lectura:
 - Se consultan nodos hasta confirmar la mayoría
 - Se puede formalizar como: $R + W > N$
 - El quórum de lectura debe ser igual a los nodos de escritura + 1: $r = w + 1$

Replicación

Motivos:

- Fiabilidad (fallo de nodo)
- Performance (tráfico alto o latencia por distancia)

En el caso del escalado geográfico, mantener las copias consistentes puede incurrir en uso de ancho de banda y volver a afectar la latencia

Clase 9 - Familia de columnas

En las BBDD tradicionales, incluso en muchas NoSQL, se almacena por fila

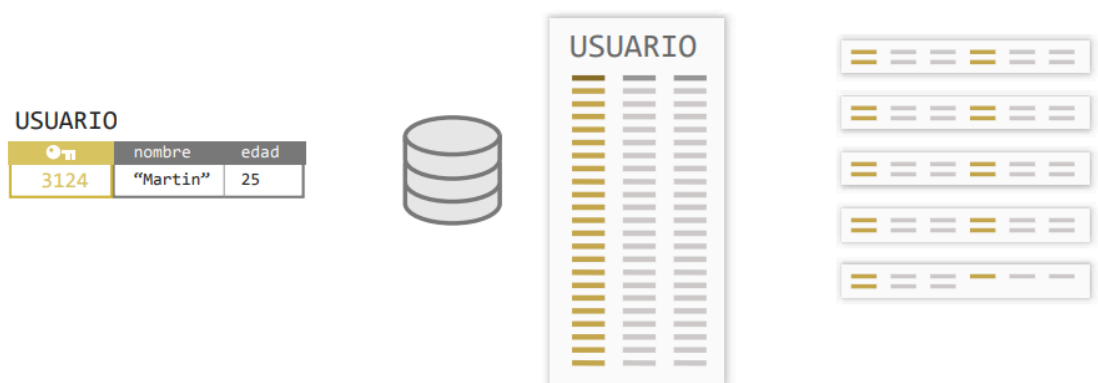
Si buscamos varias celdas de una fila, a nivel de disco que se organiza en bloques tenemos un caso de uso óptimo

Si buscamos una sola celda/columna, a nivel de disco obtenemos igualmente todas - no tan óptimo

Hay optimizaciones, pero no son parte de la lógica esencial del almacenamiento por fila

En una BBDD SQL:

- `SELECT * FROM users` → nro de llamadas I/O será más óptimo, ya que necesitamos toda la info tal cual está agrupada en disco
- `SELECT AVG(age) FROM users` → más I/O con información superflua, no necesitamos toda la info que leemos

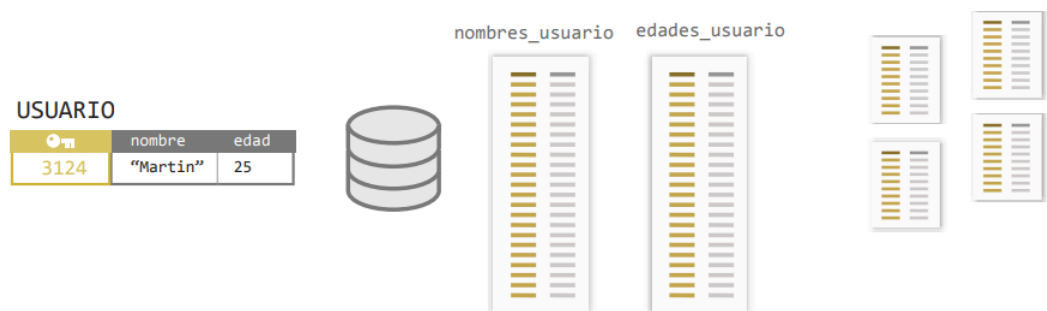


9987	3124	"Martín"	25	9988	3127	"Ana"	32	9989	3128	"Pablo"	29
9990	3130	"Santiago"	45	9991	3131	"Lorena"	46	9992	3132	"Mariano"	22
9993	3133	"Silvia"	56	9994	3134	"Juana"	21	9995	3135	"Claudio"	76

En las BBDD de familia de columnas se guarda físicamente por columna (todas las edades juntas, todos los nombres juntos, etc.), junto a su rowid

Ahora con estos ejemplos:

- `SELECT * FROM users` → nro de llamadas I/O será ineficiente, necesitamos recorrer muchos bloques para reconstruir una fila
- `SELECT AVG(age) FROM users` → muy eficiente, solo recorremos el bloque/s de la edad. Permite compresión, dado que hay un solo dato en el almacenamiento y está ordenado



3124:9987	3127:9988	3128:9989	3130:9990	3131:9991	3132:9992	3133:9993	3134:9994	3135:9995
"Martín":9987	"Ana":9988	"Pablo":9989	"Santiago":9990	"Lorena":9991	"Mariano":9992	"Silvia":9993	"Juana":9994	"Claudio":9995
25:9987	32:9988	29:9989	45:9990	46:9991	22:9992	56:9993	21:9994	76:9995

Cassandra

Cassandra promueve el "Query First Approach": se diseñan las consultas primero y los datos se desnormalizan para que cada consulta clave lea de una sola tabla; por eso no hay JOINS y una misma entidad puede replicarse en varias tablas optimizadas para distintos accesos

Ejemplo de Blog

En RDBMS:

id	name	age	lang
3124	"Martín"	25	2231
3127	"Ana"	32	2231
3128	"Pablo"	29	2231
3130	"Santiago"	45	2231
3131	"Sara"	46	2232
3132	"José"	22	2232
3133	"Franz"	56	2234
3134	"Verena"	21	2234
3135	"Helmut"	60	2234

id	name	key
2231	"Español"	sp
2232	"Portugues"	pt
2233	"English"	en
2234	"Deutsch"	de

En family column:

users_by_lang

lang	name	age
es	"Martín"	25
es	"Ana"	32
es	"Pablo"	29
es	"Santiago"	45
pt	"Sara"	46
pt	"José"	22
de	"Franz"	56
de	"Verena"	21
de	"Helmut"	60

- Partiendo de un caso de uso, modelamos la tabla detrás de la query necesaria
- La clave primaria (pk) puede ser cualquier combinación de atributos
- También hay una partition key que es parte de la pk

- La partition key indica en qué nodo/s físico se guarda cada tupla
- En este caso, “lang” pasa a ser la partition key (el resto de la pk son clustering keys usadas para ordenar). ¿Qué significa esto para las queries?
→ hay que incluir la partition key para no recorrer todos los nodos

```
CREATE TABLE available_hotels_by_city (
  city_name          VARCHAR
  date              DATE,
  hotel_id           UUID,
  hotel_name         VARCHAR,
  is_available       BOOL,
  price              FLOAT,
  PRIMARY KEY ((city_name), date, hotel_id)
);
```

La no posibilidad de usar JOIN determina el modelado

Toda consulta se diseña para buscar en una sola tabla

En consecuencia, las consultas son además más simples, no solo más veloces

Keyspace

En Cassandra, un keyspace es una BD en RDBMS

Las tablas tienen particiones, necesariamente

Prácticas asociadas a este hecho:

- Siempre se debe guardar junto lo que se recupera junto
- Evitar particiones sin límite de crecimiento (unbounded)
- Evitar particiones que concentren los reads en un momento dado



Particiones y distribución

Las claves determinan el nodo en el que se guardan los datos. Pero qué pasa si pierdo un nodo? $\Rightarrow$ necesito replicación

La partition key determina el nodo en el cual se guarda la fila

Cada nodo tiene entonces asignado parte del keyspace

Además, hay replicación entre nodos (aunque tengan espacios diferentes)

Cada nodo se asigna un número de tokens. En realidad, las partition keys pasan por una función de hash que da un token

Es P2P - cualquier nodo puede hacer todo

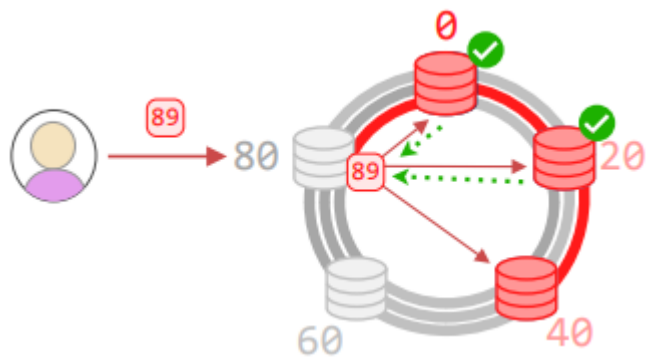
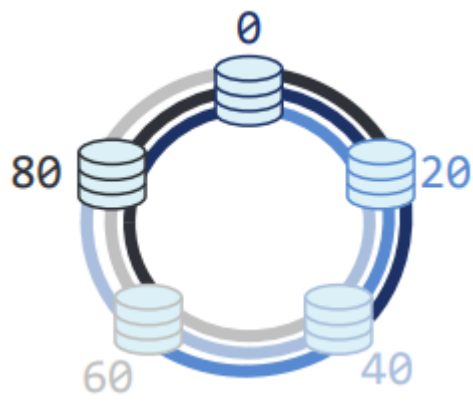
Un nodo atiende la lectura, y luego se ocupa de delegar al que tenga el dato

Se utiliza el sistema de Quorums

Cualquier nodo atiende la escritura y se asegura de que los nodos que reciben el token escriban el dato

Incluso se guarda una copia por las dudas hasta que esté confirmado

Mismas reglas de Quorum vistas antes



Clase 10 - Data warehouse

Hoy, las empresas tienen información distribuida en fuentes de datos diversas: CRM, ERP, sistemas operacionales, IoT, etc.

Un Data Warehouse viene a solucionar el problema de cómo extraer información útil para la empresa de todas esas fuentes de datos

Un Data Warehouse es un sistema que consolida, en un único repositorio, grandes volúmenes de datos de múltiples fuentes de la organización para análisis y reportes, no para el procesamiento transaccional diario. Provee una visión global, común e integrada de los datos

No es una BD, de hecho, una BD es uno de sus componentes

Características del DW:

- Integración
- Consistencia
- Variación temporal
- No volatilidad
- Organización temática.

Funciones/usos típicos:

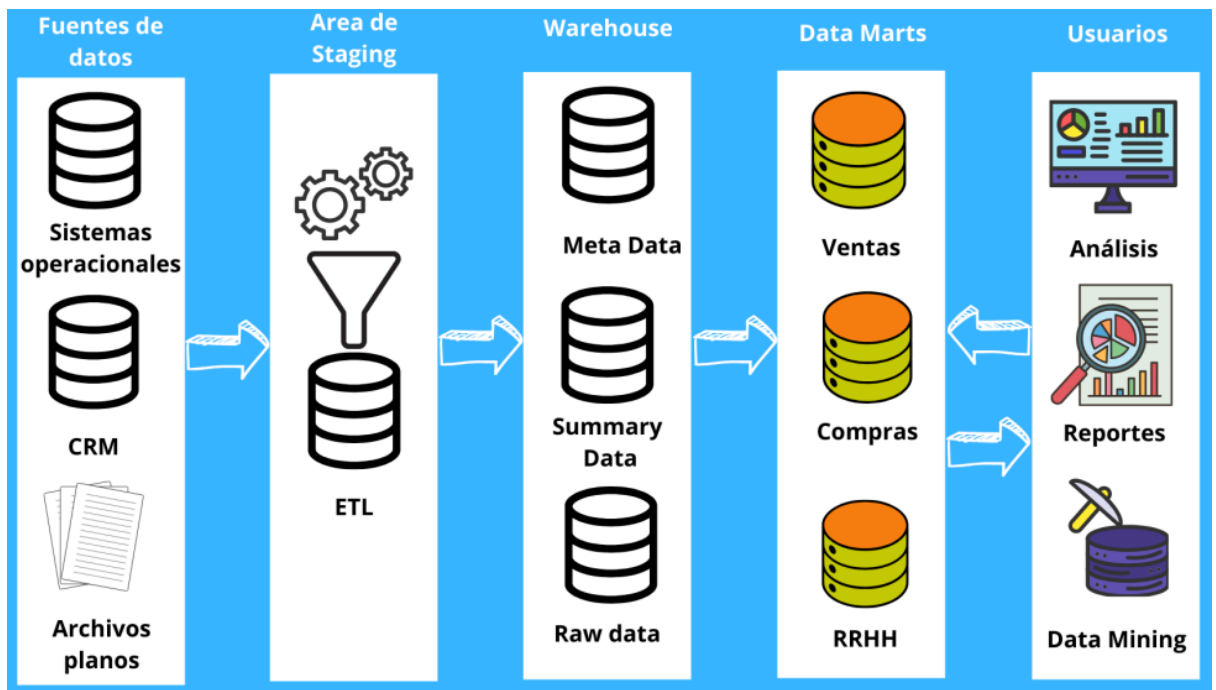
- Business intelligence
- Reportes gerenciales
- Decisiones basadas en datos
- Análisis de tendencias
- Data mining
- Machine learning

Arquitectura

Componentes:

- Fuentes de datos
- Herramientas ETL:
 - Extraction: los datos de las diversas fuentes se extraen crudos
 - Transformation: se normalizan (no en el sentido de BBDD sino de igualarlos)
 - Load

- Metadatos:
 - Son datos que describen datos
 - Ya que al tener tantas diversas fuentes de datos, es necesario poder conocer su significado y forma de interpretación
 -
- Base de datos del DW:
 - Pocas transacciones y gran volumen de datos
 - Sin updates ni deletes
- Herramientas de consulta/análisis



BBDD vs DW

Bases de Datos	Data Warehouse
Orientada a transacciones	Orientado a consultas
Operativa	Análisis y toma de decisiones estratégicas
Proceso puntual	Proceso batch masivo
Estabilidad	Dinámica
Datos desagregados	Distintos niveles de agregación
Dato actual	Evolución del dato
Usuario de perfil bajo	Usuarios de perfil alto
Datos relacionados con lo operativo	Datos relacionados con el negocio

Tablas de un DW

Hechos (medidas)

Son el objeto de los análisis y están relacionados con las dimensiones

Son tablas muy grandes y suelen estar desnormalizadas o en 3FN

A menudo incluyen diferentes agregaciones como máximo, mínimo, media, etc.

Almacena datos cuantitativos o numéricos sobre un proceso de negocio, como ventas, compras, o incidencias, para su análisis

Una tabla de hechos suele tener dos tipos de columnas:

1. Claves foráneas para tablas de dimensiones
2. Medidas, que contienen datos numéricos

Una tabla de hechos puede contener datos de hechos a nivel de detalle o agregado

Dimensiones

Representan factores por lo que se analiza un determinado área del negocio

Son pequeñas y usualmente están desnormalizadas

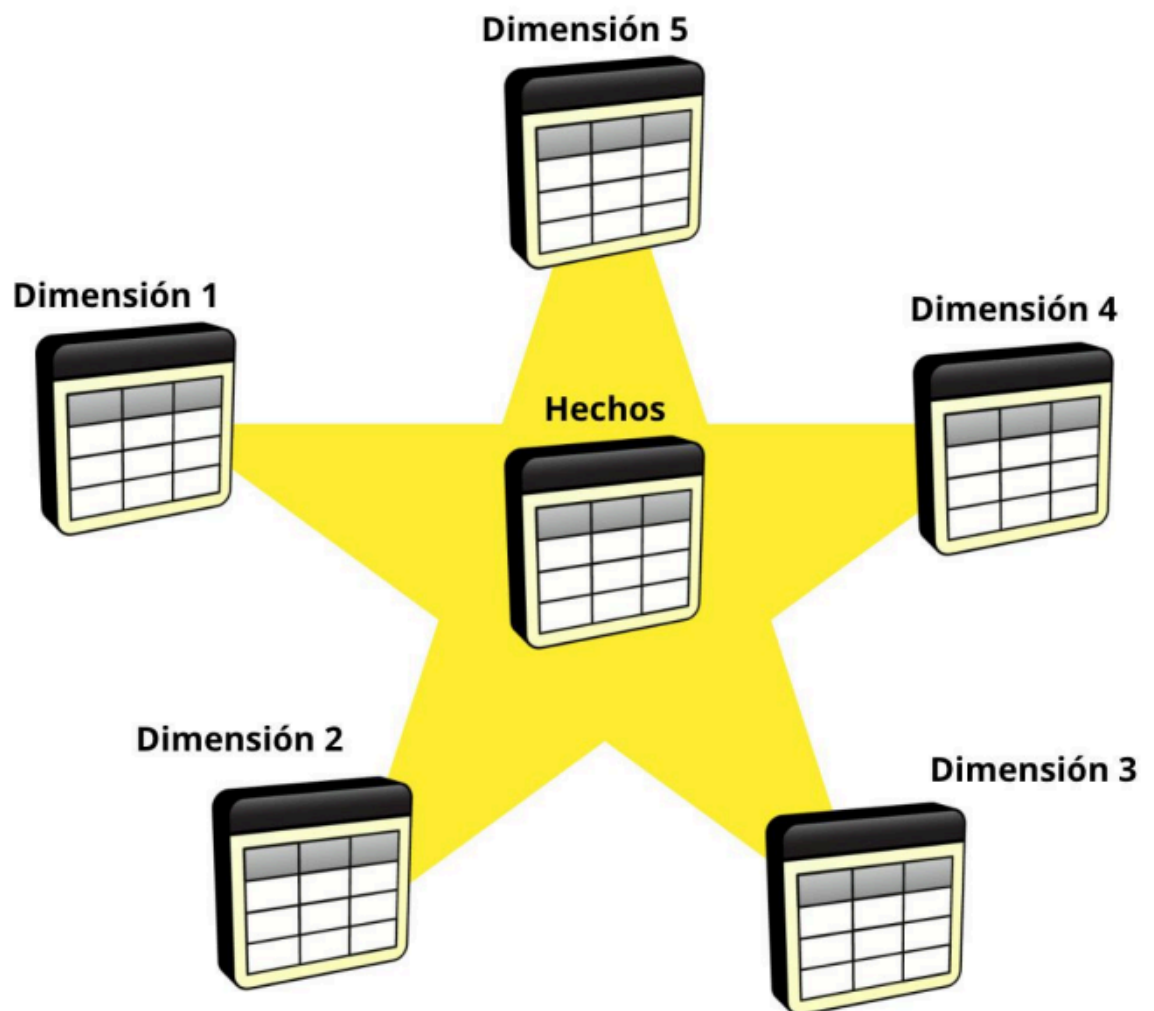
Las dimensiones contienen los metadatos sobre los hechos

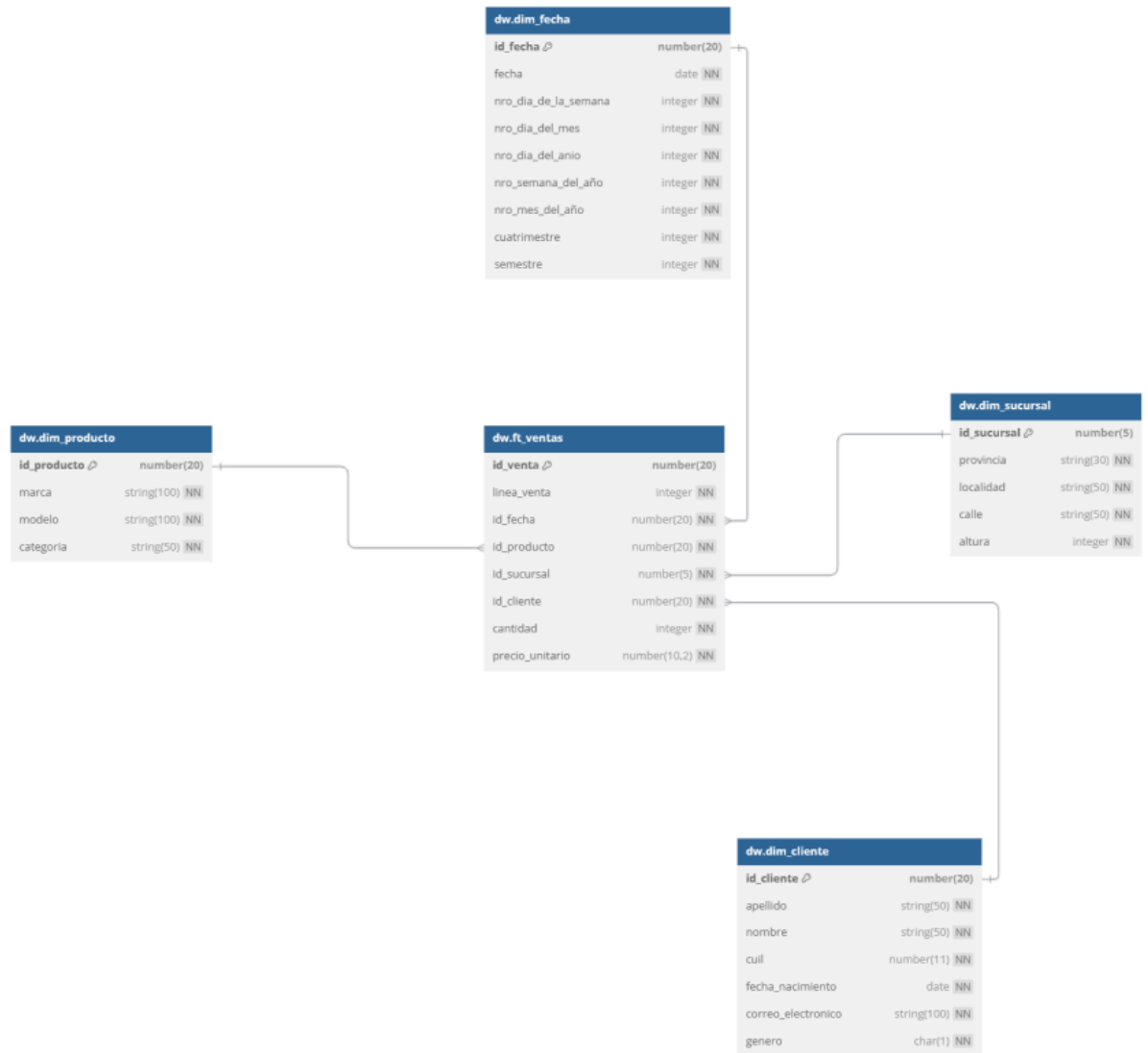
Si la información necesita disponer de varios niveles de granularidad se crean jerarquías con las dimensiones

Por ejemplo, la jerarquía fecha podría ser «día – semana – mes – trimestre – año». Las jerarquías de las dimensiones presentan relaciones n-1 de manera que un valor de un nivel sólo puede ser agrupado por un único valor de cada nivel inmediatamente superior en la jerarquía

Modelos

Estrella





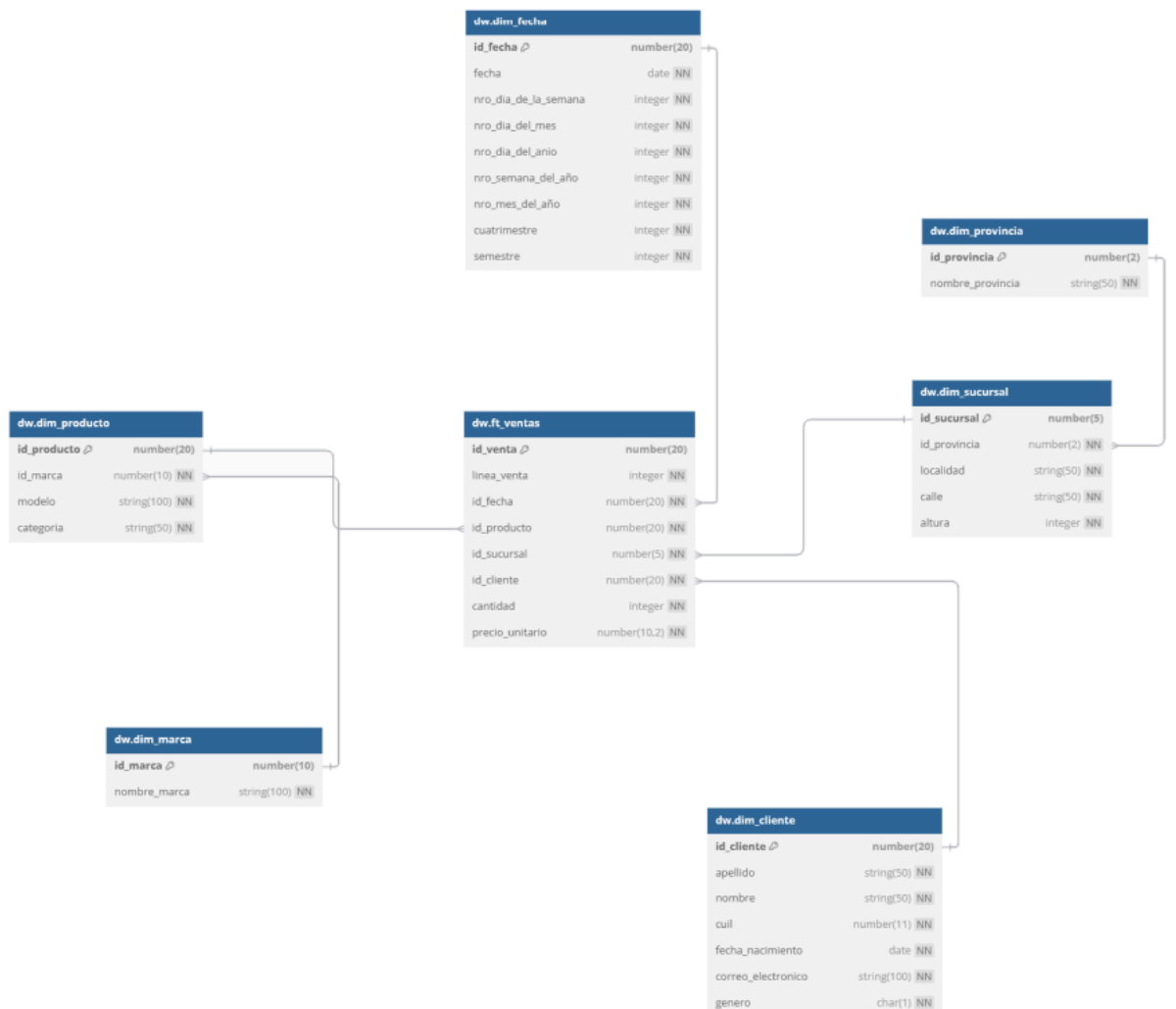
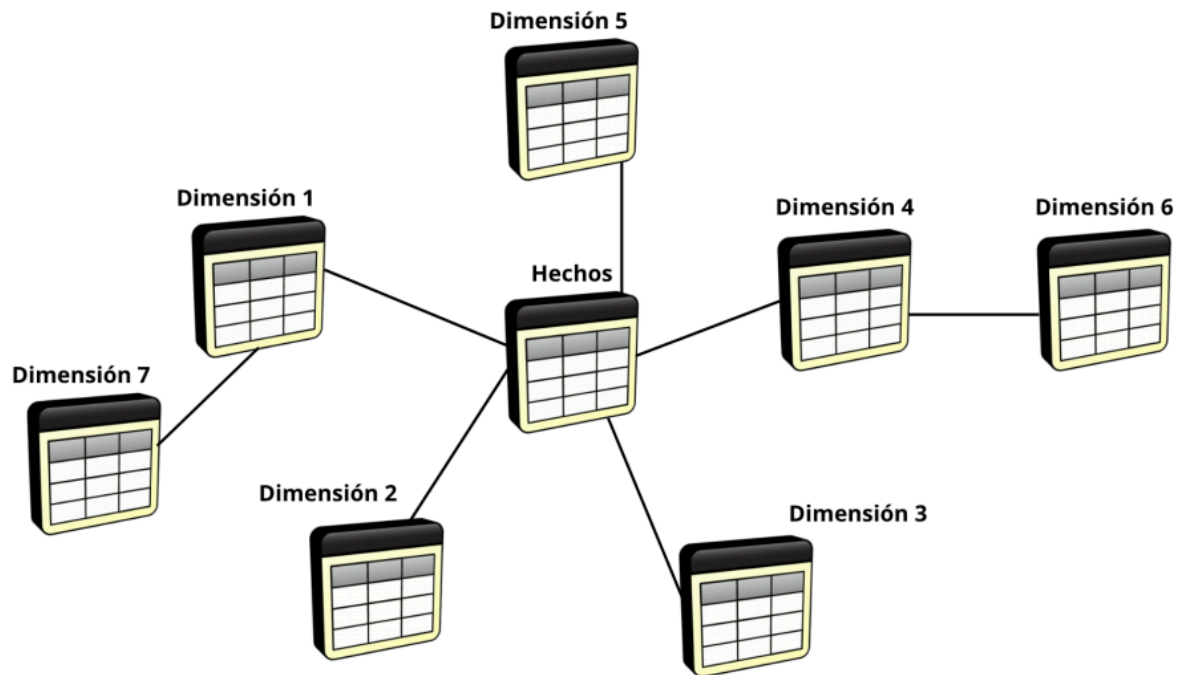
Pros:

- Es el más simple de todos los modelos, esto lo hace muy fácil de integrar en la organización, ya que es fácilmente comprensible
- Es un modelo que tiene muy buen rendimiento en las consultas, ya que solamente involucramos un join entre los datos

Contras:

- La escalabilidad puede ser peor, ya que existe un uso más ineficiente del almacenamiento
- Trabajar con tablas desnormalizadas y redundancia de datos hace que el tamaño de la tabla pudiera ser grande y se produzca un tiempo relativamente largo de carga de datos en tablas de dimensiones

Copo de nieve



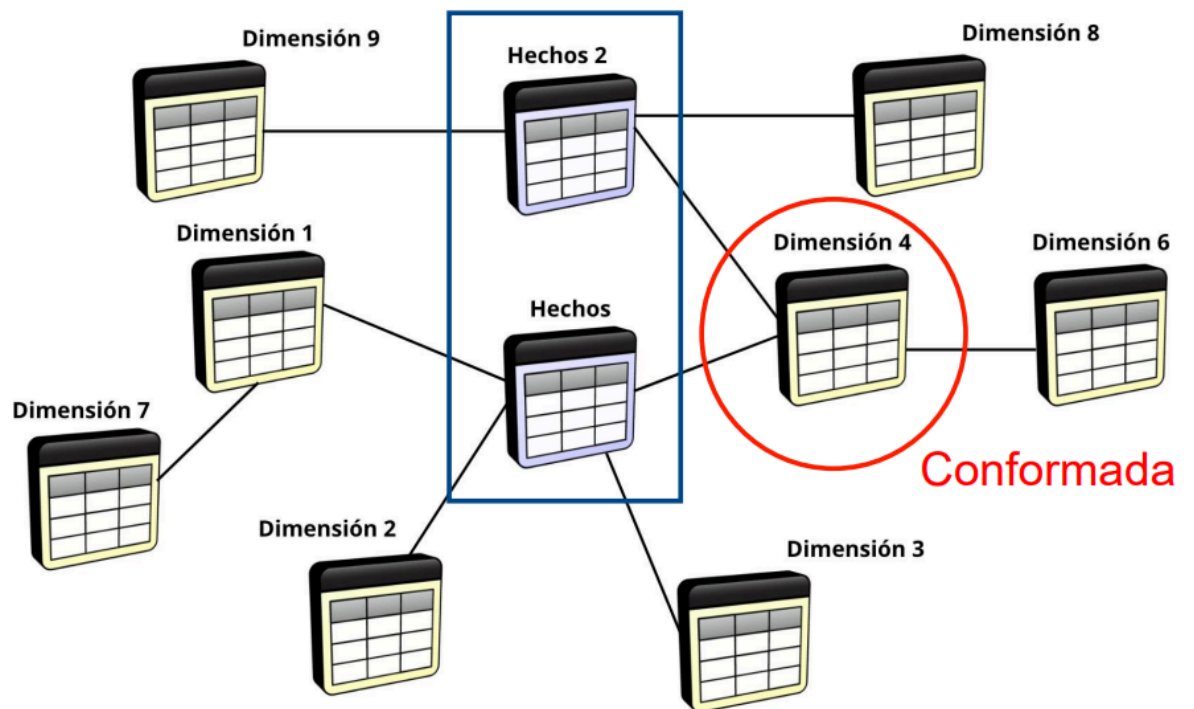
Pros:

- Permite generar más contexto alrededor de una tabla de hechos
- Trabajas con tablas normalizadas, se evita la redundancia de datos añadiendo más capas de contexto

Contras:

- El código es mucho más complejo ya que pasamos a dos niveles de join a la hora de trabajar con una dimensión

Galaxia/Constelación



Es la conjunción de modelos estrella

Varios hechos pueden compartir una misma dimensión (se llama conformada)

Data Mart

Un Data Mart es un subconjunto de los datos del Data Warehouse orientado a un área específica con el objetivo de responder a determinados análisis o necesidades de dicha área, como pueden ser ventas o finanzas

En un Data Mart independiente, los datos pueden recopilarse directamente de las fuentes

Tiene diversas funciones como organizar la información para posteriormente analizarla, realizar indicadores (KPI), evaluar los objetivos del sector que se está analizando, etc.

El objetivo es realizar un análisis detallado sobre lo que ocurre en un negocio

Data Lake

Es un repositorio orientado a almacenar gran cantidad de datos, tanto estructurados como sin estructurar

Permite almacenar los datos tal cual vienen sin necesidad de ser transformados

Ejecuta diferentes tipos de análisis, dashboards y visualizaciones, además de generar procesos de big data, análisis en tiempo real y de Machine Learning que facilitan la toma de decisiones